So sánh các bộ phân lớp tự cài đặt trên bài toán nhận dạng chữ số viết tay MNIST

Đồ án giữa kỳ, môn Nhận dạng

Khoa Toán – Tin học, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM

Trần Trinh Nhân (24280088) Lâm Nhật Tiến (24280027)

Mã nguồn: https://github.com/TranTrinhNhan1/WriteRHand Bản chạy thử: https://trantrinhnhan1.github.io/WriteRHand/

Tóm tắt

Báo cáo cài đặt và so sánh các bộ phân lớp cho bài toán nhận dạng chữ số viết tay MNIST: Gaussian Naïve Bayes, hồi quy softmax và SVM tuyến tính (Gradient Descent), MLP (lan truyền ngược), AdaBoost-SAMME và một ensemble stacking của các mô hình này. Phần tính toán lõi do nhóm tự viết, scikit-learn chỉ để đối chiếu. Siêu tham số được chọn trên tập validation; tập test chỉ dùng để báo cáo. Độ chính xác trên tập test: Naïve Bayes 81.32%, softmax 92.52%, SVM 91.95%, AdaBoost 97.60%, MLP 98.24%, stacking 99.05%. Phần nâng cao gồm CNN so với MLP (CNN đạt 98.80%), tăng cường dữ liệu, Fashion-MNIST, EMNIST Digits và một trang demo chạy trong trình duyệt.

1 Giới thiệu

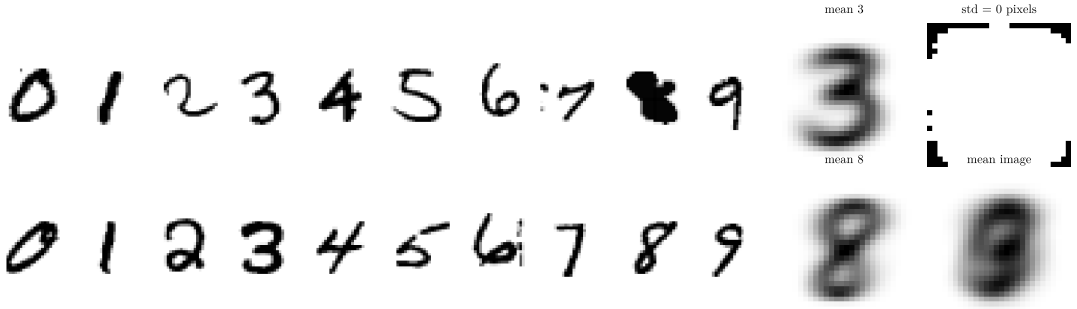
MNIST [9] gồm 70 000 ảnh xám 28×28 của các chữ số viết tay từ 0 đến 9 (60 000 ảnh huấn luyện và 10 000 ảnh kiểm thử). Đồ án cài đặt các bộ phân lớp thuộc nhiều họ khác nhau: mô hình sinh (Gaussian Naïve Bayes), mô hình tuyến tính (hồi quy softmax và SVM tuyến tính), mạng neuron nhiều lớp (MLP) và ensemble (AdaBoost-SAMME và stacking), rồi so sánh về độ chính xác, thời gian huấn luyện, thời gian dự đoán và khả năng giải thích. Phần nâng cao gồm CNN, tăng cường dữ liệu, hai bộ dữ liệu khác (Fashion-MNIST, EMNIST Digits) và giao diện demo.

Theo đề bài, phần tính toán lõi do nhóm viết bằng NumPy; scikit-learn chỉ dùng để đối chiếu, PyTorch chỉ cung cấp tensor và bộ sinh số ngẫu nhiên cho MLP và CNN (không dùng `autograd`, `nn.Module`, `optim` hay lớp tích chập dựng sẵn để huấn luyện; bảng 1). Mục 2 mô tả dữ liệu, mục 3 các phương pháp, mục 4 thí nghiệm, mục 5 kết quả, mục 6 phần nâng cao, mục 7 thảo luận và mục 9 phân công công việc.

2 Dữ liệu và tiền xử lý

Chia tập. 60 000 ảnh huấn luyện chính thức được chia phân tầng theo nhãn (seed 42) thành 50 000 ảnh train và 10 000 ảnh validation; 10 000 ảnh test chính thức được giữ riêng. Mã băm của phép chia (`52cc8206ff96f56d`) được lưu cùng mọi kết quả.

Chuẩn hóa và làm phẳng. Pixel được chia cho 255 rồi chuẩn hóa $x \leftarrow (x - \bar{x})/(s + 10^{-8})$ với $\bar{x} = 0.1306$, $s = 0.3081$ tính *chỉ trên tập train* và áp lại cho validation, test và trang demo. Ảnh được làm phẳng thành vector 784 chiều cho các mô hình không tích chập; CNN nhận ảnh $1 \times 28 \times 28$.



Hình 1. Khảo sát dữ liệu train: mẫu của mỗi chữ số, ảnh trung bình của lớp 3 và 8, các pixel có phương sai bằng 0 (màu đen) và ảnh trung bình toàn tập.

Khảo sát dữ liệu. Các lớp gần cân bằng (lớp 1 chiếm 11.24%, lớp 5 chiếm 9.03%), nên accuracy là chỉ số hợp lý; precision, recall và F1 macro vẫn được báo cáo. Khoảng 81% giá trị điểm ảnh là nền và 71 vị

Thành phần	Nhóm tự viết (NumPy)	Thư viện
Tiền xử lý, chỉ số	Chia phân tầng, chuẩn hóa, ma trận nhầm lẫn, precision/recall/F1 macro	`sklearn.metrics` chỉ trong kiểm thử
Naïve Bayes	Ước lượng tham số, log-likelihood, log-sum-exp	`GaussianNB` để đối chiếu
Softmax	Hàm mất mát, gradient giải tích, mini-batch GD	`LogisticRegression` để đối chiếu
SVM tuyến tính	Hàm mất mát hinge, đạo hàm dưới, mini-batch GD	`LinearSVC` để đối chiếu
MLP	Lan truyền tiến và ngược, khởi tạo He, dropout, SGD có momentum, dừng sớm	Tensor, `matmul`, bộ sinh số ngẫu nhiên của PyTorch; `autograd` và `optim` chỉ trong kiểm thử
AdaBoost-SAMME	Cập nhật SAMME, cây CART có trọng số dựng bằng histogram	`AdaBoostClassifier` để đối chiếu
CNN	im2col/col2im, tích chập, gộp cực đại, gradient của từng lớp	Như MLP; `F.conv2d`, `F.max_pool2d` chỉ trong kiểm thử
Stacking	Đặc trưng cấp 1, meta-learner softmax, chọn λ bằng kiểm định chéo	–

Bảng 1. Phạm vi tự cài đặt.

trí ở viền luôn bằng nền (hình 1); các pixel này có phương sai bằng 0 nên cần làm trơn phương sai trong Naïve Bayes. Tương quan trung bình giữa hai pixel kề nhau trong cùng một lớp là 0.65, nên giả thiết độc lập của Naïve Bayes không đúng với dữ liệu này.

3 Phương pháp

Ký hiệu: $\boldsymbol{x} \in \mathbb{R}^D$ ($D = 784$) là ảnh đã chuẩn hóa và làm phẳng, $y \in \{0, \dots, K-1\}$ ($K = 10$) là nhãn, tập huấn luyện gồm N cặp $(\boldsymbol{x}_i, y_i)$. Bảng 1 nêu rõ phần nào do nhóm viết và phần nào dùng thư viện.

3.1 Gaussian Naïve Bayes

Mô hình sinh giả thiết các pixel độc lập có điều kiện theo lớp và mỗi pixel có phân phối chuẩn [2, 6]:

$$\hat{y} = \arg\max_k \Big[\log \pi_k - \tfrac{1}{2} \sum_{j=1}^{D} \Big(\log(2\pi\sigma_{kj}^2) + \frac{(x_j - \mu_{kj})^2}{\sigma_{kj}^2} \Big) \Big]. \tag{1}$$

Ước lượng hợp lý cực đại cho $\pi_k = N_k/N$, μ_{kj} và σ_{kj}^2 là trung bình và phương sai mẫu của pixel j trong lớp k. Pixel luôn bằng 0 có phương sai 0 làm $\log \sigma^2 \to -\infty$, nên mọi phương sai được cộng $\varepsilon = \texttt{var_smoothing} \cdot \max_j \mathrm{Var}(x_j)$, cùng quy ước với scikit-learn. Phép tính thực hiện trong không gian log và chuẩn hóa hậu nghiệm bằng log-sum-exp để tránh underflow khi nhân 784 thừa số. Huấn luyện cần một lượt qua dữ liệu, $O(ND)$; dự đoán một ảnh $O(KD)$. Siêu tham số duy nhất là `var_smoothing`. Giả thiết độc lập thường được dùng dù không đúng [5], nhưng ở đây nó sai rõ rệt (mục 2).

3.2 Hồi quy softmax (logistic đa lớp)

Với $\boldsymbol{z} = W^\top \boldsymbol{x} + \boldsymbol{b}$ và $p_k = e^{z_k} / \sum_m e^{z_m}$, hàm mất mát trên mini-batch $\mathcal{B}$ gồm cross-entropy và phạt ℓ_2 (không phạt bias):

$$L(W, \boldsymbol{b}) = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \log p_{y_i}(\boldsymbol{x}_i) + \frac{\lambda}{2} \|W\|_F^2. \tag{2}$$

Vì $\partial L / \partial z_k = p_k - \mathbb{1}[y = k]$, với $X \in \mathbb{R}^{B \times D}$, ma trận xác suất P và ma trận one-hot Y:

$$\nabla_W L = \tfrac{1}{B} X^\top (P - Y) + \lambda W, \qquad \nabla_{\boldsymbol{b}} L = \tfrac{1}{B} \mathbf{1}^\top (P - Y). \tag{3}$$

Softmax và cross-entropy được tính ở dạng ổn định số (trừ $\max_k z_k$, dùng $\log \sum \exp$). Cập nhật $\theta \leftarrow \theta - \eta \nabla_\theta L$ theo mini-batch có xáo trộn mỗi epoch; hàm mục tiêu lồi nên khởi tạo bằng 0. Mô hình tuyến tính nên biên quyết định giữa hai lớp là một siêu phẳng trong không gian pixel.

3.3 SVM tuyến tính

Bộ phân lớp tuyến tính thứ hai dùng hàm mất mát hinge của SVM [2]. Với mỗi lớp k có một điểm số $z_k = \boldsymbol{w}_k^\top \boldsymbol{x} + b_k$ (một bộ phân lớp một-chống-còn-lại) và nhãn $t_{ik} = +1$ nếu $y_i = k$, ngược lại $t_{ik} = -1$. Trên mini-batch $\mathcal{B}$:

$$L(W, \boldsymbol{b}) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \sum_{k=1}^{K} \max\left(0,\, 1 - t_{ik} z_{ik}\right) + \frac{\lambda}{2} \|W\|_F^2. \tag{4}$$

Hàm hinge không khả vi tại biên, nên dùng đạo hàm dưới: $\partial L / \partial z_{ik} = -t_{ik} \mathbb{1}[t_{ik} z_{ik} < 1] / B$, rồi $\nabla_W L = X^\top (\partial L / \partial Z) + \lambda W$ và $\nabla_{\boldsymbol{b}} L = \mathbf{1}^\top (\partial L / \partial Z)$, cập nhật $\theta \leftarrow \theta - \eta \nabla_\theta L$ như softmax. Dự đoán là $\arg\max_k z_k$. Biến thể hinge bình phương $\max(0, 1 - tz)^2$ (khả vi) cũng được cài đặt và được thử trong lượt quét. Hệ số λ tương ứng với $1/(Cn)$ trong ký hiệu thường dùng của SVM. SVM chỉ cho điểm số chứ không cho xác suất.

3.4 Perceptron nhiều lớp và lan truyền ngược

Các lớp ẩn tính $\boldsymbol{z}_l = \boldsymbol{a}_{l-1} W_l + \boldsymbol{b}_l$, $\boldsymbol{a}_l = \mathrm{ReLU}(\boldsymbol{z}_l) \odot \boldsymbol{m}_l$ với $\boldsymbol{a}_0 = \boldsymbol{x}$; lớp cuối cho logits và softmax; mất mát là cross-entropy trung bình. Trọng số khởi tạo theo He [8], $W_l \sim \mathcal{N}(0, 2/\text{fan_in})$. Dropout dạng *inverted* [13] dùng mặt nạ $m_{l,j} \in \{0, 1/(1-p)\}$, chỉ bật khi huấn luyện. Lan truyền ngược [12] được viết tay:

$$\boldsymbol{\delta}_L = \tfrac{1}{B}(\mathrm{softmax}(\boldsymbol{z}_L) - Y), \quad \nabla_{W_l} = \boldsymbol{a}_{l-1}^\top \boldsymbol{\delta}_l, \quad \nabla_{\boldsymbol{b}_l} = \textstyle\sum_{\text{hàng}} \boldsymbol{\delta}_l, \quad \boldsymbol{\delta}_{l-1} = (\boldsymbol{\delta}_l W_l^\top) \odot \mathbb{1}[\boldsymbol{a}_{l-1} > 0] \odot \boldsymbol{m}_{l-1}. \tag{5}$$

Cập nhật dùng SGD có momentum [14]: $\boldsymbol{v} \leftarrow \mu \boldsymbol{v} + \boldsymbol{g}$, $\theta \leftarrow \theta - \eta \boldsymbol{v}$. Dừng sớm [11] dựa trên 8% tập train tách riêng bên trong mỗi lần huấn luyện (kiên nhẫn 5 epoch, khôi phục trọng số tốt nhất), nên tập validation không tham gia huấn luyện. Thuật toán 1 tóm tắt một epoch.

Thuật toán 1. Một epoch huấn luyện MLP (`src/models/mlp.py`)

1: Xáo trộn chỉ số của tập train, chia thành các mini-batch kích thước B.
2: Với mỗi mini-batch: lan truyền tiến, lưu $\boldsymbol{z}_l, \boldsymbol{a}_l, \boldsymbol{m}_l$ của mọi lớp; tính cross-entropy.
3: Tính $\boldsymbol{\delta}_L$ rồi lùi từ lớp cuối về lớp đầu theo công thức (5) để thu $\nabla_{W_l}, \nabla_{\boldsymbol{b}_l}$.
4: Cập nhật $\boldsymbol{v}_l \leftarrow \mu \boldsymbol{v}_l + \nabla_l$, $\theta_l \leftarrow \theta_l - \eta \boldsymbol{v}_l$.
5: Cuối epoch: tính loss trên tập dừng sớm (tắt dropout); nếu không cải thiện quá 10^{-4} trong 5 epoch thì dừng và khôi phục trọng số tốt nhất.

3.5 AdaBoost-SAMME với cây quyết định có trọng số

Boosting là ensemble thực sự: các bộ học yếu được huấn luyện tuần tự và mỗi bộ tập trung vào mẫu mà các bộ trước phân lớp sai. SAMME [17] mở rộng AdaBoost [7] sang K lớp. Khởi tạo $w_i = 1/N$; ở vòng t, huấn luyện cây h_t với trọng số mẫu w, đặt $\mathrm{err}_t = \sum_i w_i \mathbb{1}[h_t(\boldsymbol{x}_i) \neq y_i]$ (với $\sum_i w_i = 1$) và

$$\begin{gathered} \alpha_t = \nu\Big(\ln \frac{1 - \mathrm{err}_t}{\mathrm{err}_t} + \ln(K-1)\Big), \qquad w_i \leftarrow \frac{w_i \, e^{\alpha_t \mathbb{1}[h_t(\boldsymbol{x}_i) \neq y_i]}}{\sum_{i'} w_{i'} e^{\alpha_t \mathbb{1}[h_t(\boldsymbol{x}_{i'}) \neq y_{i'}]}}, \\ H(\boldsymbol{x}) = \arg\max_k \sum_t \alpha_t \mathbb{1}[h_t(\boldsymbol{x}) = k]\,, \end{gathered} \tag{6}$$

với ν là hệ số co. Số hạng $\ln(K-1)$ chỉ yêu cầu bộ học yếu tốt hơn đoán ngẫu nhiên ($\mathrm{err} < 1 - 1/K$); vòng lặp dừng khi điều kiện này bị vi phạm. Bộ học yếu là cây CART độ sâu tối đa d giảm tạp chất Gini có trọng số [3]. Để huấn luyện được trên $50\,000 \times 784$, mỗi đặc trưng được lượng tử hóa một lần thành n_{bins} khoảng đều nhau và trọng số theo (nút, đặc trưng, bin, lớp) được cộng dồn bằng histogram, nên mọi ngưỡng của mọi đặc trưng được đánh giá trong $O(ND)$ mỗi tầng; ngưỡng thực được lưu cho dự đoán. Mỗi cây chỉ xét một tập con ngẫu nhiên các đặc trưng (`max_features`); Thuật toán 2 tóm tắt quy trình. Chi phí huấn luyện $O(T\,d\,N\,D)$ với T cây và dự đoán $O(T\,d)$; điểm số đầu ra là tỉ lệ phiếu có trọng số chứ không phải xác suất đã hiệu chỉnh.

Thuật toán 2. AdaBoost-SAMME (`src/models/adaboost.py`)

1: Lượng tử hóa X thành bin (một lần); $w_i \leftarrow 1/N$.
2: Với $t = 1, \ldots, T$: dựng cây h_t từ histogram có trọng số, chọn ở mỗi nút cặp (đặc trưng, ngưỡng) làm giảm tạp chất nhiều nhất; dừng khi không giảm hoặc đạt độ sâu d.
3: $\mathrm{err}_t \leftarrow \sum_i w_i \mathbb{1}[h_t(\boldsymbol{x}_i) \neq y_i]$; nếu $\mathrm{err}_t \geq 1 - 1/K$ thì dừng.

4: Tính α_t và cập nhật, chuẩn hóa w theo công thức (6).
5: Dự đoán: $\hat{y} = \arg\max_k \sum_t \alpha_t \mathbb{1}[h_t(\boldsymbol{x}) = k]$.

3.6 Mạng tích chập nhỏ

CNN [9] khác MLP ở chỗ mỗi đơn vị chỉ nhìn một cửa sổ cục bộ và các cửa sổ dùng chung trọng số. Cấu hình được chọn trên validation là hai khối tích chập 3×3 với 16 và 32 kênh, mỗi khối theo sau bởi ReLU và gộp cực đại 2×2, rồi hai lớp dày đặc:

$$1{\times}28{\times}28 \to 16{\times}26{\times}26 \to 16{\times}13{\times}13 \to 32{\times}11{\times}11 \to 32{\times}5{\times}5 \to 800 \to 64 \to 10.$$

Các kích thước này được đọc từ `CNN.describe()` của mô hình đã huấn luyện, tổng cộng 56 714 tham số (MLP: 535 818). Tích chập được cài bằng *im2col*: mỗi cửa sổ $k \times k \times C_{\text{in}}$ được trải thành một hàng nên đầu ra là tích ma trận patches $\cdot\, W$; gradient của kernel là patches$^\top \partial L/\partial$out và gradient theo đầu vào được cộng dồn bằng *col2im*. Gộp cực đại ghi lại vị trí cực đại để chuyển gradient về đúng ô. Huấn luyện dùng cùng SGD momentum, khởi tạo He, dropout và dừng sớm như MLP.

3.7 Stacking

Stacking [15] là ensemble học cách kết hợp các mô hình đã huấn luyện thay vì dùng một quy tắc cố định. Mô hình cấp 0 là sáu mô hình ở trên (Naïve Bayes, softmax, SVM, MLP, AdaBoost, CNN), mỗi mô hình đã được huấn luyện trên tập train. Với mỗi ảnh, đặc trưng cấp 1 là $\log \max(p_k, 10^{-4})$ của từng mô hình có xác suất, hoặc điểm số z_k của SVM; ghép lại được $K \times 6 = 60$ đặc trưng, chuẩn hóa theo trung bình và độ lệch chuẩn. Mô hình cấp 1 (meta-learner) là hồi quy softmax của mục 3.2. Meta-learner được huấn luyện trên *tập validation* (hold-out đối với các mô hình cơ sở), không phải trên tập train: đầu ra của mô hình trên chính dữ liệu huấn luyện quá tự tin (AdaBoost đạt 100%), nên meta-learner học ở đó sẽ chỉ tin vào mô hình khớp dữ liệu nhất. Hệ số ℓ_2 của meta-learner chọn bằng kiểm định chéo 5 phần *bên trong* tập validation. Thuật toán 3 tóm tắt quy trình.

Thuật toán 3. Stacking với hold-out (`src/experiments/stacking.py`)

1: Nạp các mô hình cơ sở đã huấn luyện trên tập train; tính đặc trưng cấp 1 F của tập validation.
2: Với mỗi λ trong lưới, đo accuracy kiểm định chéo 5 phần của meta-learner trên (F, y_{val}); chọn λ cao nhất.
3: Huấn luyện meta-learner trên toàn bộ (F, y_{val}) với λ đã chọn.
4: Dự đoán ảnh mới: chạy cả sáu mô hình cơ sở, ghép đặc trưng, đưa vào meta-learner.

4 Thí nghiệm

Chọn siêu tham số. Mỗi siêu tham số được quét lần lượt (từng tham số một, các tham số khác giữ ở giá trị tốt nhất hiện có) và giá trị cho validation accuracy cao nhất được chọn. Công cụ tìm kiếm do nhóm tự viết (`src/experiments/search.py`), không dùng Optuna. Mỗi lượt quét chính là thí nghiệm "đổi một siêu tham số, giữ nguyên phần còn lại" của đề bài (hình 4); cấu hình được chọn ở mục A.

Huấn luyện cuối và chỉ số. Cấu hình được chọn được huấn luyện lại trên 50 000 ảnh train (seed 42), rồi báo cáo accuracy, macro precision, macro recall, macro F1 và ma trận nhầm lẫn trên train, validation và test. Với ma trận nhầm lẫn C, $\text{TP}_k = C_{kk}$, $\text{FP}_k = \sum_i C_{ik} - \text{TP}_k$, $\text{FN}_k = \sum_j C_{kj} - \text{TP}_k$; precision, recall và F_1 tính theo từng lớp rồi lấy trung bình không trọng số.

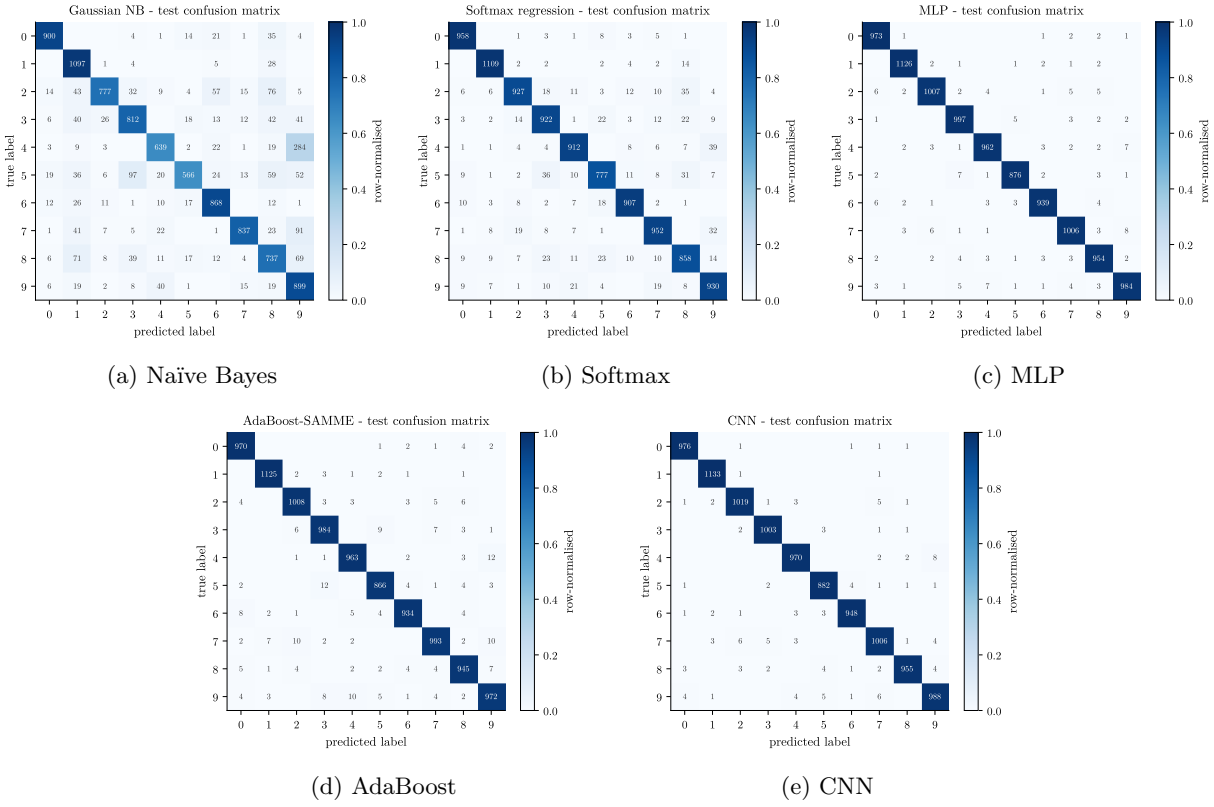
Đo thời gian. Thời gian đo trên CPU, không dùng GPU (13th Gen Intel Core i7-13620H, 16 luồng logic, PyTorch 8 luồng, Python 3.12.14): huấn luyện là thời gian `fit` trên 50 000 ảnh, dự đoán là trung vị của 5 lần dự đoán 10 000 ảnh test, độ trễ là trung vị của 200 lần dự đoán một ảnh. Thời gian chỉ nên so ở mức bậc độ lớn vì có thể chênh khi máy chạy đồng thời việc khác.

Đối chiếu với scikit-learn. Naïve Bayes, softmax, SVM, MLP và AdaBoost được huấn luyện lại bằng scikit-learn (SVM: `LinearSVC` với $C = 1/(\lambda n)$) với cùng siêu tham số và cùng dữ liệu. Ngoài ra bộ kiểm thử tự động (121 kiểm thử) kiểm tra tham số và xác suất của Naïve Bayes khớp `GaussianNB`, gradient của softmax, SVM (hinge bình phương), MLP và CNN khớp sai phân hữu hạn và autograd của PyTorch trên mạng nhỏ, và tích chập im2col khớp cài đặt ngây thơ.

Tái lập. Mọi kết quả được ghi thành tệp JSON trong `results/`; bảng và hình của báo cáo được sinh từ các tệp đó. Lệnh chạy lại nằm trong `README.md` và notebook `notebooks/reproduce_results.ipynb`. Các lần chạy được ghi bằng Weights & Biases [1] ở chế độ offline.

	Train				Validation				Test			
Mô hình	Acc	P	R	F1	Acc	P	R	F1	Acc	P	R	F1
Gaussian NB	80.13	81.85	79.74	79.98	79.67	81.39	79.27	79.50	81.32	82.98	80.95	81.16
Softmax regression	93.62	93.56	93.53	93.54	92.15	92.07	92.05	92.05	92.52	92.45	92.41	92.42
MLP	99.87	99.87	99.87	99.87	98.18	98.17	98.17	98.17	98.24	98.23	98.23	98.23
AdaBoost-SAMME	100.00	100.00	100.00	100.00	97.59	97.59	97.57	97.58	97.60	97.58	97.58	97.58
CNN	99.40	99.40	99.39	99.39	98.65	98.65	98.63	98.64	98.80	98.79	98.79	98.79
Linear SVM	92.53	92.43	92.43	92.42	91.38	91.27	91.26	91.25	91.95	91.85	91.84	91.82
Stacking	99.83	99.83	99.83	99.83	99.12	99.12	99.11	99.12	99.05	99.05	99.04	99.04

Bảng 2. Accuracy, precision, recall và F1 macro (%) trên train, validation và test (MNIST, seed 42). Hai hàng cuối là SVM tuyến tính và stacking; với stacking, validation là dữ liệu huấn luyện của meta-learner nên không phải ước lượng sạch.



(a) Naïve Bayes (b) Softmax (c) MLP

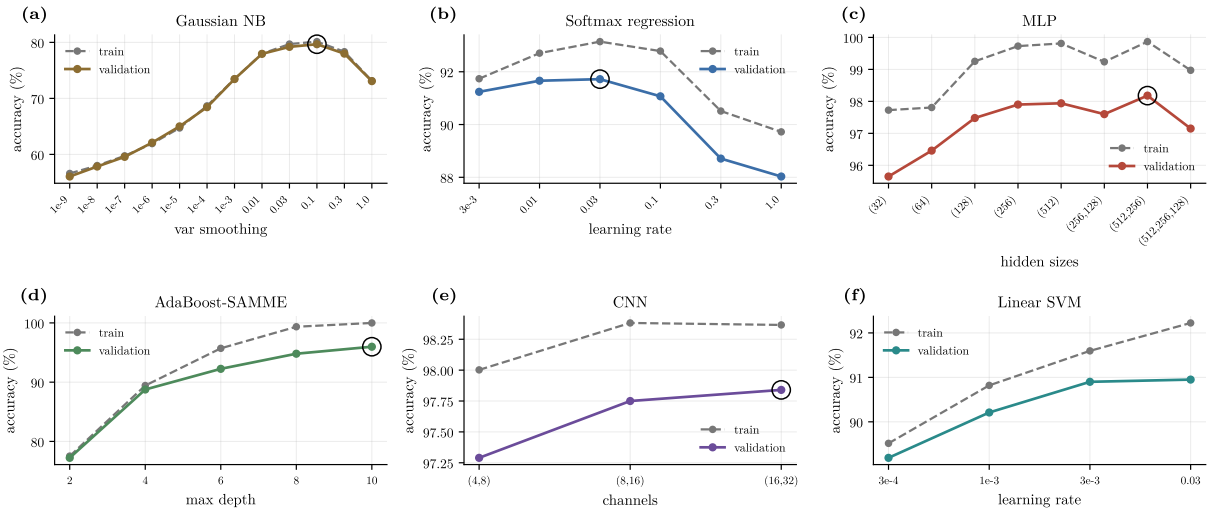
(d) AdaBoost (e) CNN

Hình 2. Ma trận nhầm lẫn trên tập test (hàng: chữ số thật; cột: dự đoán). Ma trận của train và validation nằm trong `results/final/*.json`.

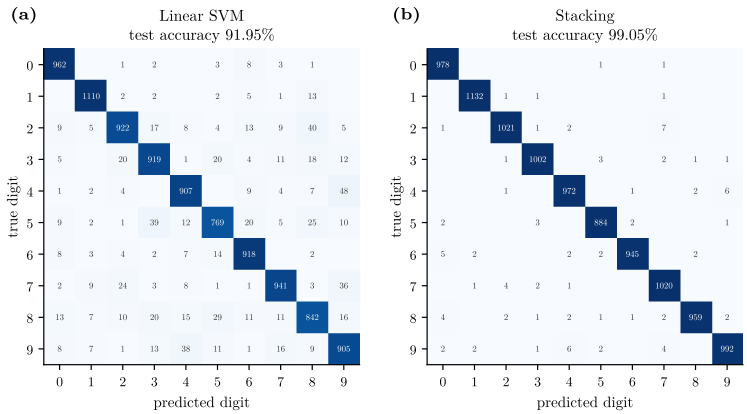
5 Kết quả

5.1 Độ chính xác trên ba tập

Bảng 2 cho thứ tự trên tập test: MLP (98.24%) và AdaBoost (97.60%) vượt xa softmax (92.52%) và Naïve Bayes (81.32%). F1 macro lệch accuracy không quá 0.16 điểm vì các lớp gần cân bằng. AdaBoost đạt 100% trên train còn MLP đạt 99.87%, nên hai mô hình này khớp rất sát dữ liệu huấn luyện (chênh train–test 2.40 và 1.63 điểm). Softmax chỉ đạt 93.62% trên train, tức bị giới hạn bởi biên tuyến tính. Naïve Bayes có test cao hơn train (−1.19 điểm). SVM tuyến tính (91.95%) gần softmax, cũng bị giới hạn bởi biên tuyến tính (train 92.53%). Stacking đạt 98.69% (kiểm định chéo trên validation) và 99.05% trên test, cao hơn CNN, mô hình đơn tốt nhất, 0.25 điểm (mục 5.4). Mỗi mô hình chỉ được huấn luyện một lần nên khoảng cách giữa MLP và AdaBoost (0.64 điểm) chưa đủ để kết luận mô hình nào tốt hơn.



Hình 4. Accuracy train và validation theo siêu tham số quan trọng nhất của từng mô hình; vòng tròn đánh dấu giá trị được chọn. Các lượt quét chạy tuần tự nên phần còn lại của cấu hình ở giá trị tốt nhất tìm được trước đó.



Hình 3. Ma trận nhầm lẫn trên tập test của SVM tuyến tính (a) và stacking (b).

Cả năm mô hình đều nhầm nhiều nhất ở những cặp có nét chung (hình 2): $4 \to 9$ là cặp lớn nhất của Naïve Bayes (284 trong 1868 lỗi), softmax, AdaBoost và CNN; MLP nhầm nhiều nhất $7 \to 9$. Các cặp $5 \to 3$ và $7 \to 9$ cũng nằm trong ba cặp đầu của nhiều mô hình. SVM nhầm nhiều nhất $4 \to 9$ (48 ảnh), $2 \to 8$ và $5 \to 3$; stacking chỉ còn 95 ảnh sai (hình 3), nhiều nhất là $2 \to 7$ (7 ảnh).

5.2 Ảnh hưởng của siêu tham số

Hình 4 cho thấy: (a) Naïve Bayes: `var_smoothing` tăng từ 10^{-9} lên 0.1 làm validation tăng từ 56.05% lên 79.67%, train và validation gần trùng nhau nên mô hình bị giới hạn bởi giả thiết độc lập chứ không do overfitting; (b) softmax: tốc độ học 0.03 tốt nhất, giá trị lớn hơn làm cả train lẫn validation giảm; (c) MLP: hai lớp ẩn $(512, 256)$ đạt 98.18%, hơn một lớp ẩn (97.94%) và ba lớp ẩn (97.15%); (d) AdaBoost: validation tăng đơn điệu theo độ sâu cây; (e) CNN: $(16, 32)$ kênh cho validation cao nhất; (f) SVM: tốc độ học 0.01 tốt nhất và hàm hinge (91.16%) cao hơn hinge bình phương (89.24%). Với AdaBoost (độ sâu 10) và CNN, giá trị tốt nhất nằm ở mép khoảng đã quét nên có thể chưa tối ưu. Số epoch của SVM (75) lấy từ đường cong validation. Số epoch của MLP do dừng sớm quyết định (dừng ở epoch 14, khôi phục epoch 9), số vòng boosting là 393.

5.3 Thời gian huấn luyện, thời gian dự đoán và đối chiếu scikit-learn

AdaBoost đắt nhất (bảng 3): 817.9 s huấn luyện và 18.705 ms cho một ảnh, vì phải duyệt 393 cây (một phần do vòng lặp Python trong cài đặt). Naïve Bayes, softmax và SVM rẻ nhất (SVM: 11.5 s huấn luyện). Với stacking, thời gian huấn luyện là tổng thời gian huấn luyện sáu mô hình cơ sở cộng 1.0 s của meta-learner, và mỗi ảnh phải chạy qua cả sáu mô hình nên chậm hơn AdaBoost một chút. CNN chậm hơn MLP khoảng 13 lần khi huấn luyện và 7 lần khi dự đoán.

Mô hình	Huấn luyện (s)	Dự đoán 10 000 ảnh (s)	Một ảnh (ms)	Tệp xuất (KiB)
Gaussian NB	0.4	0.226	0.076	62
Softmax regression	12.7	0.024	0.010	31
MLP	11.0	0.051	0.042	2 094
AdaBoost-SAMME	817.9	0.751	18.705	9 457
CNN	143.0	1.163	0.315	223
Linear SVM	11.5	0.023	0.008	–
Stacking	997.5	2.204	23.789	–

Bảng 3. Chi phí trên CPU: huấn luyện trên 50 000 ảnh, dự đoán 10 000 ảnh test, thời gian một ảnh và kích thước tệp mô hình xuất ra (SVM và stacking chưa xuất sang web).

Mô hình	Test (ours)	Test (sklearn)	$\lvert\Delta\rvert$	Fit ours (s)	Fit sklearn (s)
Gaussian NB	81.32	81.32	0.00	0.4	0.2
Softmax regression	92.52	91.86	0.66	12.7	154.6
MLP	98.24	98.02	0.22	11.0	57.8
AdaBoost-SAMME	97.60	97.50	0.10	817.9	1892.8

Bảng 4. Bản tự cài đặt và scikit-learn với cùng siêu tham số: accuracy test (%), hiệu tuyệt đối và thời gian huấn luyện.

Naïve Bayes trùng khớp với scikit-learn (bảng 4). SVM (hinge) của nhóm đạt 91.95% so với 92.02% của `LinearSVC` cùng hàm mất mát ($C = 0.2$), chênh 0.07 điểm; `LinearSVC` cần 99.0 s và dừng ở giới hạn vòng lặp mà chưa hội tụ. Ba mô hình còn lại lệch từ 0.10 đến 0.66 điểm: softmax của nhóm cao hơn vì scikit-learn (L-BFGS) chạm giới hạn vòng lặp mà chưa hội tụ, còn nhóm dừng ở epoch chọn bằng validation; MLP chênh 0.22 điểm; AdaBoost chênh 0.10 điểm, hai bản dùng cách chia nút khác nhau (histogram so với ngưỡng chính xác) nên thời gian không so sánh trực tiếp được.

5.4 Stacking

Mô hình	Accuracy test (%)	Trọng số $\lvert W\rvert$ TB
Gaussian NB	81.32	0.072
Softmax regression	92.52	0.077
MLP	98.24	0.167
AdaBoost-SAMME	97.60	0.119
CNN	98.80	0.240
Linear SVM	91.95	0.078
Bỏ phiếu mềm (không học)	97.92	–
Stacking	99.05	–

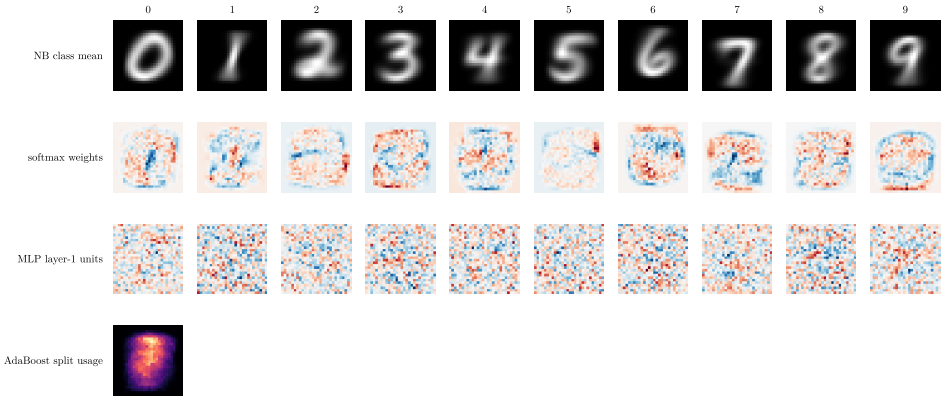
Bảng 5. Accuracy test của các mô hình cơ sở, của bỏ phiếu mềm và của stacking; cột cuối là trị tuyệt đối trung bình (TB) của trọng số meta-learner ứng với từng mô hình cơ sở (đặc trưng đã chuẩn hóa). Bỏ phiếu mềm lấy trung bình xác suất của các mô hình có xác suất, không học gì thêm.

Meta-learner chọn $\lambda = 0.0001$ bằng kiểm định chéo 5 phần trên validation, cho accuracy kiểm định chéo 98.69%. Trên test, stacking đạt 99.05% (bảng 5), cao hơn CNN 0.25 điểm và cao hơn rõ rệt phép bỏ phiếu mềm không học (97.92%, thấp hơn cả CNN). Số lỗi giảm từ 120 (CNN) xuống 95 ảnh. Meta-learner đặt trọng số lớn nhất cho CNN, MLP và AdaBoost. Các lưu ý: (1) mức tăng 0.25 điểm trên 10 000 ảnh test là một lần chạy một seed, chưa đo biến thiên; (2) meta-learner được huấn luyện trên chỉ 10 000 ảnh validation; (3) accuracy train (99.83%) và validation (99.12%) của stacking không phải ước lượng sạch vì validation là dữ liệu của meta-learner; (4) chi phí suy luận là tổng của sáu mô hình.

5.5 So sánh ưu và nhược điểm

Mô hình	Độ chính xác	Thời gian huấn luyện và dự đoán	Khả năng giải thích
Naïve Bayes	Thấp nhất (81.32%): giả thiết độc lập bị mâu thuẫn bởi tương quan giữa các pixel kề nhau.	Rẻ: 0.4 s huấn luyện, 0.076 ms mỗi ảnh.	Tham số μ_{kj}, σ_{kj} là ảnh trung bình và phương sai theo lớp, đọc trực tiếp.
Softmax	92.52%; train chỉ 93.62%, bị giới hạn bởi biên tuyến tính.	Dự đoán nhanh (0.010 ms mỗi ảnh); huấn luyện 12.7 s.	Trọng số theo lớp vẽ được thành ảnh; $x_i W_{ik}$ phân rã chính xác điểm số.
SVM tuyến tính	91.95%; gần softmax, cùng giới hạn của biên tuyến tính.	Rẻ: 11.5 s huấn luyện, 0.008 ms mỗi ảnh.	Như softmax: trọng số theo lớp vẽ được thành ảnh; điểm số không phải xác suất.
MLP	98.24%; chênh train–test 1.63 điểm.	11.0 s huấn luyện, 0.042 ms mỗi ảnh.	Khó: đơn vị ẩn gần như không đọc được bằng mắt.
AdaBoost-SAMME	97.60%, gần MLP; train 100%.	Đắt nhất: 817.9 s huấn luyện, 18.705 ms mỗi ảnh.	Bản đồ tần suất pixel được cây dùng; điểm là tỉ lệ phiếu, chưa phải xác suất.
Stacking	Cao nhất (99.05%), hơn CNN 0.25 điểm.	Đắt: huấn luyện sáu mô hình cơ sở, 23.8 ms mỗi ảnh.	Khó nhất: kết hợp sáu mô hình; chỉ biết mô hình nào được trọng số lớn (CNN, MLP).

Bảng 6. Ưu và nhược điểm của các mô hình theo bốn tiêu chí của đề bài (số liệu từ bảng 2 và 3; nhận xét về khả năng giải thích là định tính, hình 5).



Hình 5. Tham số hoặc thống kê của từng mô hình: ảnh trung bình theo lớp (Naïve Bayes), trọng số theo lớp (softmax; đỏ dương, xanh âm), các đơn vị ẩn đầu tiên của MLP và tần suất pixel được dùng để chia nhánh (AdaBoost).

6 Phần nâng cao

Các thí nghiệm dưới đây dùng cùng mã, giao thức và bộ đánh giá như mục 5.

6.1 CNN so với MLP

CNN gồm hai khối tích chập 3×3 (16 và 32 kênh), mỗi khối có ReLU và gộp cực đại 2×2, rồi hai lớp dày đặc (mục 3.6). Siêu tham số được chọn bằng cùng quy trình quét trên validation.

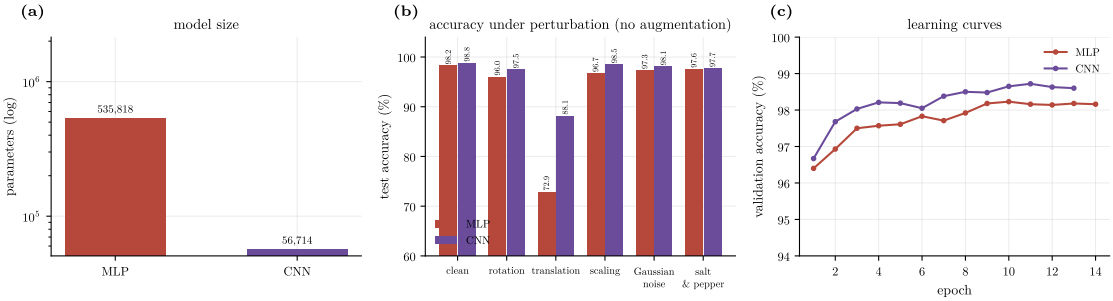
CNN đạt 98.80% so với 98.24% của MLP với số tham số ít hơn 9.4 lần, và chênh train–test nhỏ hơn (0.60 so với 1.63 điểm). Khác biệt rõ nhất khi ảnh test bị dịch 3 pixel: MLP còn 72.86%, CNN còn 88.14% (hình 6b). Cái giá là thời gian: huấn luyện lâu hơn khoảng 13 lần và mỗi ảnh chậm hơn khoảng 7 lần (bảng 3).

6.2 Tăng cường dữ liệu

Năm phép biến đổi chỉ áp dụng cho tập train, nhãn không đổi (hình 7): xoay trong $[-15°, 15°]$, dịch trong $[-2, 2]$ pixel, co giãn trong $[0.9, 1.1]$, nhiễu Gauss $\sigma = 0.1$ và nhiễu muối tiêu 2%. Mỗi điều kiện (gốc, từng phép riêng lẻ, nhóm hình học, cả năm phép) giữ nguyên siêu tham số và chỉ thêm một bản sao biến đổi của

	MLP	CNN
Số tham số	535 818	56 714
Accuracy test (%)	98.24	98.80
Accuracy train − test (điểm)	1.63	0.60
Thời gian huấn luyện (s)	11	143
Độ trễ một ảnh (ms)	0.042	0.315

Bảng 7. CNN và MLP trên MNIST, cùng giao thức. Accuracy khi ảnh test bị biến đổi nằm ở hình 6b.



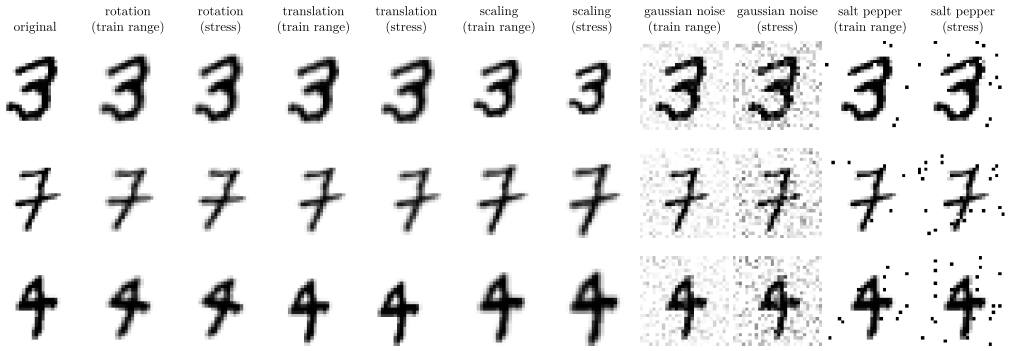
Hình 6. CNN và MLP: số tham số (a), accuracy khi ảnh test bị biến đổi cố định, huấn luyện không tăng cường (b) và validation accuracy theo epoch (c).

mỗi ảnh train (100 000 ảnh); chạy 3 seed (MLP) hoặc 2 seed (CNN). Điều kiện được chọn theo validation; để đo độ bền, tập test được biến đổi mạnh hơn lúc huấn luyện. Vì điều kiện tăng cường có dữ liệu gấp đôi điều kiện gốc, ảnh hưởng của loại biến đổi và của lượng dữ liệu bị lẫn.

	MLP (3 seed)		CNN (2 seed)	
Điều kiện	Δ val (điểm)	Test (%)	Δ val (điểm)	Test (%)
Không tăng cường	–	98.02	–	98.82
Xoay	+0.57 ± 0.47	98.48	+0.23 ± 0.01	99.17
Dịch chuyển	+0.54 ± 0.55	98.41	+0.22 ± 0.03	99.17
Co giãn	+0.42 ± 0.45	98.33	+0.20 ± 0.01	99.06
Nhiễu Gauss	+0.38 ± 0.42	98.35	+0.19 ± 0.20	99.06
Nhiễu muối tiêu	+0.37 ± 0.40	98.32	+0.17 ± 0.06	99.00
Hình học (xoay, dịch, co giãn)	+0.31 ± 0.33	98.24	+0.25 ± 0.16	99.20
Cả năm phép	+0.09 ± 0.40	98.11	+0.31 ± 0.01	99.25

Bảng 8. Tăng cường dữ liệu với cấu hình đã chọn: Δ là hiệu validation accuracy so với điều kiện gốc *cùng seed* (trung bình ± độ lệch chuẩn mẫu của hiệu), "Test" là accuracy test sạch trung bình trên các seed. Mỗi điều kiện tăng cường dùng 100 000 ảnh train, điều kiện gốc 50 000.

Bảng 8 cho thấy với MLP, mọi điều kiện tăng validation 0.09–0.57 điểm nhưng cùng bậc với độ lệch



Hình 7. Ảnh gốc và từng phép biến đổi ở mức huấn luyện và mức kiểm tra độ bền.

Mô hình	MNIST	Fashion-MNIST	EMNIST Digits
Gaussian NB	81.32 / 81.16	67.27 / 66.03	80.03 / 80.13
Softmax regression	92.52 / 92.42	84.27 / 84.18	93.81 / 93.81
MLP	98.24 / 98.23	88.59 / 88.55	98.36 / 98.36
AdaBoost-SAMME	97.60 / 97.58	84.74 / 84.88	97.38 / 97.38
CNN	98.80 / 98.79	88.49 / 88.48	98.64 / 98.64

Bảng 9. Accuracy / macro-F1 trên tập test (%) của từng bộ dữ liệu; mỗi bộ dữ liệu có phép chia train/validation/test riêng, siêu tham số chọn riêng trên validation của bộ đó (quét rút gọn cho Fashion-MNIST và EMNIST Digits).

Mô hình	Huấn luyện trên MNIST	Huấn luyện trên EMNIST Digits
Gaussian NB	49.80	80.03
Softmax regression	53.59	93.81
MLP	76.70	98.36
CNN	92.15	98.64
AdaBoost-SAMME	73.69	97.38

Bảng 10. Accuracy (%) trên 40 000 ảnh test của EMNIST Digits. Cột giữa: mô hình huấn luyện trên MNIST; cột phải: mô hình huấn luyện trên EMNIST Digits (hai mô hình khác nhau).

chuẩn ghép cặp (0.33–0.55), nên chưa khẳng định được tác dụng trên ảnh sạch; với CNN, test sạch tăng từ 98.82% lên 99.00–99.25% (chỉ 2 seed). Tác dụng rõ nhất là độ bền với đúng loại biến đổi: huấn luyện với phép dịch nâng accuracy của MLP trên ảnh bị dịch từ 72.1% lên 94.5%. Một phép tăng cường cũng có thể làm giảm độ bền ở loại khác: với CNN, chỉ dùng phép dịch hoặc nhóm hình học làm accuracy trên ảnh nhiễu Gauss mạnh giảm từ 97.2% xuống 95.0% và 93.6%.

6.3 Fashion-MNIST và EMNIST Digits

Fashion-MNIST [16] gồm ảnh 28×28 của 10 loại quần áo; EMNIST Digits [4] gồm chữ số viết tay (240 000 ảnh train, 40 000 ảnh test; nhóm lấy 60 000 ảnh train chia 50 000/10 000 và giữ nguyên 40 000 ảnh test). Mỗi bộ dữ liệu có phép chia, hằng số chuẩn hóa và siêu tham số riêng (quét rút gọn, một seed).

Thứ hạng ổn định ở hai bộ chữ số (Naïve Bayes $<$ softmax $<$ AdaBoost $<$ MLP $\leq$ CNN, bảng 9) nhưng đổi ở Fashion-MNIST: MLP và CNN ngang nhau (88.59 và 88.49%) và Naïve Bayes giảm còn 67.27%. Mô hình huấn luyện trên MNIST rồi đánh giá trên EMNIST Digits (cùng 10 chữ số) đều giảm (bảng 10): CNN giảm ít nhất (6.5 điểm), softmax nhiều nhất (40.2 điểm). Fashion-MNIST dùng nhãn khác nên không có phép kiểm tra chéo hợp lệ với bộ chữ số. Nhóm chưa thử bộ chữ số viết tay tiếng Việt.

6.4 Giao diện demo

Trang web (React, TypeScript, ONNX Runtime Web [10]) cho phép vẽ hoặc chọn ảnh, chọn bộ dữ liệu và một trong năm mô hình rồi xem dự đoán chạy ngay trong trình duyệt. Mô hình được xuất sang ONNX (riêng AdaBoost dùng tệp nhị phân của các cây) và đối chiếu với mô hình Python: với 15 mô hình xuất ra, sai lệch xác suất lớn nhất giữa ONNX (hoặc bộ đánh giá nhị phân) và mô hình Python là 1.9×10^{-4} và độ trùng nhãn thấp nhất là 100.00% (trên 2 000 ảnh test cho mỗi mô hình). Mô hình nằm trên Hugging Face Hub, được ghim theo mã commit. Chức năng "nhìn vào bên trong" vẽ lại phép tính của chính mô hình đã triển khai; đó là trực quan hóa phép tính, không phải lời giải thích vì sao mô hình quyết định như vậy.

7 Thảo luận

Độ chính xác và độ phức tạp. Hai nhóm mô hình tách biệt rõ: Naïve Bayes, softmax và SVM tuyến tính (81.32%, 92.52% và 91.95%) thấp hơn MLP, AdaBoost và CNN ít nhất 5 điểm, và nguyên nhân khác nhau: Naïve Bayes bị giới hạn bởi giả thiết độc lập (tương quan giữa pixel kề nhau là 0.65), softmax và SVM bị giới hạn bởi biên tuyến tính (train chỉ 93.62% và 92.53%); hàm mất mát hinge thay cho cross-entropy gần như không đổi kết quả. MLP và AdaBoost cho độ chính xác gần nhau; MLP nhanh hơn nhiều (11.0 s so với 817.9 s huấn luyện, 0.042 ms so với 18.705 ms mỗi ảnh). CNN cho độ chính xác cao nhất trong các mô hình đơn với ít tham số hơn MLP nhưng chậm hơn trong cài đặt NumPy trên CPU. Stacking cao hơn CNN 0.25 điểm nhưng tốn thời gian huấn luyện và suy luận của cả sáu mô hình cơ sở.

Khả năng giải thích. Tham số của Naïve Bayes và softmax đọc được dưới dạng ảnh (hình 5); đơn vị ẩn của MLP gần như không đọc được bằng mắt; tần suất chia nhánh của AdaBoost tập trung ở vùng trung tâm ảnh. Đây là nhận xét định tính về tham số, không giải thích vì sao một ảnh cụ thể bị phân vào một lớp.

8 Kết luận

Nhóm đã tự cài đặt các bộ phân lớp Gaussian Naïve Bayes, softmax, SVM tuyến tính, MLP, AdaBoost-SAMME, một ensemble stacking và một CNN nhỏ, đánh giá trên cả train, validation và test với accuracy, precision, recall, F1 macro và ma trận nhầm lẫn, và đối chiếu với scikit-learn. Trên MNIST, stacking đạt 99.05%, CNN 98.80%, MLP 98.24%, AdaBoost 97.60%, SVM 91.95%, softmax 92.52% và Naïve Bayes 81.32%. AdaBoost gần MLP về độ chính xác nhưng đắt hơn nhiều về thời gian. Tăng cường dữ liệu chủ yếu cải thiện độ bền với đúng loại biến đổi. Kết quả chỉ áp dụng cho các bộ dữ liệu đã đo, không suy ra cho chữ viết tay nói chung.

9 Phân công công việc và công bố sử dụng AI

Phân công. Bảng dưới ghi các việc kiểm chứng được từ lịch sử Git của kho mã; tỉ lệ đóng góp do hai thành viên tự thống nhất.

Thành viên	MSSV	Công việc có bằng chứng trong kho mã	Tỉ lệ
Lâm Nhật Tiến (GitHub: Tien-Lam28)	24280027	23 commit và PR #1–#4: dựng kho, README, tải và tiền xử lý dữ liệu, lớp cha, độ đo, `Evaluator`, Gaussian NB, bản đầu của MLP, EDA, notebook; được phân công Naïve Bayes và MLP.	50%
Trần Trinh Nhân (GitHub: TranTrinhNhan1)	24280088	Khởi tạo kho; được phân công mô hình tuyến tính, ensemble và phần mở rộng. Giai đoạn hoàn thiện (softmax, AdaBoost-SAMME, tái cấu trúc MLP và độ đo, bộ kiểm thử, pipeline thực nghiệm, CNN, các thí nghiệm mở rộng, xuất mô hình, website, báo cáo, slide) do thành viên này thực hiện.	50%

Bảng 11. Phân công và tỉ lệ đóng góp.

Công bố sử dụng AI. Nhóm có dùng trợ lý AI để hỗ trợ lập trình và điều chỉnh siêu tham số. Các thành viên chịu trách nhiệm về mã nộp.

Tài liệu tham khảo

[1] Lukas Biewald. Experiment tracking with Weights and Biases, 2020. Software available from `https://www.wandb.com/`.

[2] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[3] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

[4] Gregory Cohen, Saeed Afshar, Jonathan Tapson, and André van Schaik. EMNIST: an extension of MNIST to handwritten letters, 2017. arXiv:1702.05373.

[5] Pedro Domingos and Michael Pazzani. On the optimality of the simple Bayesian classifier under zero-one loss. *Machine Learning*, 29(2–3):103–130, 1997.

[6] Richard O. Duda, Peter E. Hart, and David G. Stork. *Pattern Classification*. Wiley, 2 edition, 2001.

[7] Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.

[8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.

[9] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to

document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[10] ONNX Runtime developers. ONNX Runtime, 2021. https://onnxruntime.ai/.

[11] Lutz Prechelt. Early stopping – but when? In *Neural Networks: Tricks of the Trade*, volume 1524 of *Lecture Notes in Computer Science*, pages 55–69. Springer, 1998.

[12] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[13] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.

[14] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, volume 28 of *PMLR*, pages 1139–1147, 2013.

[15] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.

[16] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms, 2017. arXiv:1708.07747.

[17] Ji Zhu, Hui Zou, Saharon Rosset, and Trevor Hastie. Multi-class AdaBoost. *Statistics and Its Interface*, 2(3):349–360, 2009.

A Cấu hình được chọn

Mô hình	Siêu tham số được chọn	Số cấu hình đã chạy
Gaussian NB	`var_smoothing=0.1`	12
Softmax regression	`learning_rate=0.03, l2=0.0, epochs=67, batch_size=128`	16
MLP	`hidden_sizes=(512, 256), learning_rate=0.05, momentum=0.9, p_drop=0.0, batch_size=128, epochs=40`	22
AdaBoost-SAMME	`n_estimators=393, max_depth=10, learning_rate=0.1, n_bins=32, max_features=0.5`	14
CNN	`channels=(16, 32), kernel_size=3, dense=(64,), learning_rate=0.01, momentum=0.9, p_drop=0.2, batch_size=128, epochs=20`	10

Bảng 12. Cấu hình chọn bằng validation. `epochs` của softmax và `n_estimators` của AdaBoost lấy từ đường cong validation của một lần chạy dài. Toàn bộ các lượt quét nằm trong `results/tuning`.